

FPL Analytics App — Step-by-Step Build Guide (Angular Edition)

Since you're using Cursor, treat Cursor as both:

Your AI software architect

Your AI implementation engineer

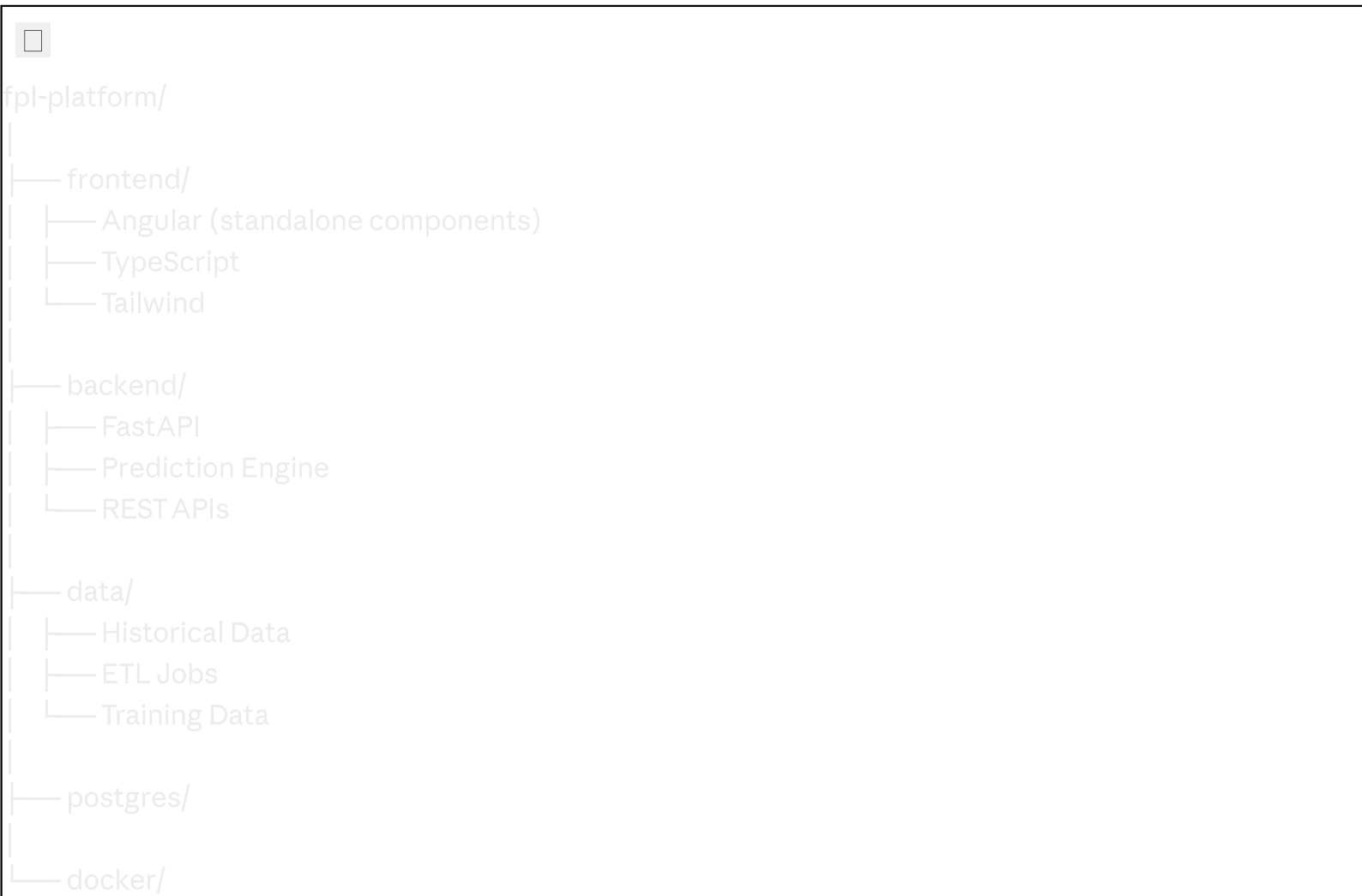
The biggest mistake most people make is opening Cursor and saying:

"Build me an FPL app"

This usually results in a messy codebase.

Instead, you're going to make Cursor build the project in carefully controlled phases.

Goal Architecture



Phase 0: Install Everything

Starting from absolutely nothing.

Install Git Download: <https://git-scm.com/downloads> Verify: `git --version`

Install Node.js LTS Install: <https://nodejs.org> Verify: `node -v` / `npm -v`

Install Angular CLI Install: `npm install -g @angular/cli` Verify: `ng version`

Install Python Install Python 3.12+: <https://www.python.org/downloads/> Verify: `python --version`

Install Docker Desktop Install: <https://www.docker.com/products/docker-desktop/> Verify: `docker --version`

Install PostgreSQL Install: <https://www.postgresql.org/download/> Or later run PostgreSQL through Docker (recommended).

Phase 1: Create Project Workspace

Create folder: `FPL-Analytics`

Open it in Cursor.

Phase 2: Let Cursor Create Initial Structure

Open Cursor Agent.

Prompt:

You are the lead architect of an FPL analytics platform.

Create a production-ready monorepo structure with:

- frontend (Angular 18+ standalone app, TypeScript strict mode)
- backend (FastAPI)
- PostgreSQL database
- Docker support
- Environment variable management
- README

Do not implement features yet.

Only create folder structure and setup files.

Accept changes.

Commit:

```
git init
git add .
git commit -m "initial architecture"
```

Phase 3: Setup Frontend

Scaffold Angular first, then let Cursor wire up the rest:

```
ng new frontend --routing --style=scss --ssr=false --standalone
```

Cursor Prompt:

```
Set up this Angular frontend with:

- Standalone components (no NgModules)
- Angular Router
- Tailwind CSS
- Angular Material or PrimeNG for base UI components
- @tanstack/angular-query-experimental for server-state/data fetching
- ng2-charts (Chart.js) or ngx-charts for data visualization
- Signals for local component state

Generate commands and files needed.
```

Run generated commands.

Expected structure:

```
frontend/
├── src/app
│   ├── components
│   ├── pages
│   ├── services
│   ├── models
│   └── core
```

Commit.

Phase 4: Setup Backend

Cursor Prompt:



```
Create FastAPI backend structure with:  
  
- routers  
- services  
- database  
- models  
- schemas  
- prediction  
- tests  
  
Use SQLAlchemy.  
  
Do not implement routes yet.
```

Expected:



```
backend/  
  
├── app  
│   ├── routers  
│   ├── services  
│   ├── models  
│   ├── schemas  
│   ├── prediction  
│   └── database
```

Commit.

Phase 5: Setup PostgreSQL

Recommended: Docker.

Create: `docker-compose.yml`

Cursor Prompt:



Create docker compose for:

- postgres
- backend
- frontend

Include volume persistence.

Run: `docker compose up -d`

Verify database starts.

Phase 6: Create Database Schema

Before generating code, design schema.

Cursor Prompt:



Design a normalized PostgreSQL schema for:

players
teams
fixtures
gameweeks
player_match_stats
player_season_stats
predictions

Include foreign keys and indexes.

Review carefully.

Then instruct: "Generate SQLAlchemy models."

Commit.

Phase 7: Load Historical Data

This is where the real value starts.

Use: `vaastav/Fantasy-Premier-League` — historical FPL data repository.

Create folder: `data/`

Cursor Prompt:



Create ETL scripts to:

1. Download historical FPL data
2. Normalize data
3. Insert into PostgreSQL

Store scripts inside data/pipelines.

Expected:



data/
└── pipelines/

Phase 8: Ingestion Layer

Build scheduled updates.

Prompt:



Create ingestion services that fetch:

bootstrap-static
fixtures
event live data

from official FPL API.

Store data in PostgreSQL.

Create endpoints:

sync/players
sync/fixtures
sync/gameweek

Test manually.

Phase 9: Build APIs

Now create the API layer.

Prompt:



Create REST API endpoints for:

GET /players
GET /players/{id}
GET /teams
GET /fixtures
GET /predictions

Include pagination and filtering.

Cursor should generate: routers, services, DTOs.

Commit.

Phase 10: Build Frontend Pages

One page at a time. Avoid asking Cursor to build the entire app.

Player List

Prompt:



Create a player explorer page (standalone Angular component).

Features:

- data table (Angular Material Table or TanStack Table Angular adapter)
- search
- filters
- sorting

Connect to backend API via a PlayersService using Angular's HttpClient and @tanstack/angular-query-experimental for caching.

Player Profile

Prompt:



Create a player profile page.

Show:

- season stats
- price history
- fixtures
- charts

Use ng2-charts (Chart.js) or ngx-charts.

Team Page

Prompt:



Create a team page with:

- squad
- fixtures
- statistics

Phase 11: Analytics Layer

Create feature engineering.

Prompt:



Create feature generation service.

Calculate:

- last 3 game average
- last 5 game average
- rolling xG
- rolling xA
- home away form
- opponent strength

Store results: player_features table.

Phase 12: First Prediction Model

Keep it simple.

Prompt:


Create machine learning pipeline.

Input:

- recent form
- price
- minutes
- opponent strength
- home away

Target:

next gameweek points

Start with linear regression.

Phase 13: Prediction API

Prompt:


Create prediction service that generates:

expected_points
confidence

for every player.

Create: GET /predictions

Phase 14: Top Picks Dashboard

Prompt:



Create a dashboard page showing:

Top Predicted Players

Best Captains

Best Differentials

Best Budget Options

Sort by expected points.

Phase 15: Upgrade the Model

When V1 works:

Prompt:



Replace linear regression with XGBoost.

Create evaluation metrics:

MAE

RMSE

R2

Persist models to disk.

Cursor Workflow You Should Follow Daily

For every feature:

Step 1 — Create requirement. Example: Build fixture difficulty feature.

Step 2 — Ask Cursor: "Write a technical design document." Review.

Step 3 — Ask: "Break implementation into tasks."

Step 4 — Ask: "Implement task 1 only."

Step 5 — Review. Commit.



git add .

git commit -m "feature completed"

Best Cursor Rule File

Create `.cursorrules`:



You are a senior staff engineer.

Rules:

1. Never make breaking changes.
2. Follow existing architecture.
3. Explain changes before implementation.
4. Generate tests.
5. Use TypeScript strict mode.
6. Follow Angular style guide and SOLID principles.
7. Prefer standalone components over NgModules.
8. Prefer small commits.
9. Document every public API.
10. Never create duplicate logic.
11. Ask for clarification in comments if assumptions are required.

What I Would Personally Build First

Week 1: Database, historical data import, player API

Week 2: Player explorer, player profile, charts

Week 3: Feature engineering, prediction model

Week 4: Top picks dashboard, deployment

By the end of Month 1 you'll already have a usable FPL research platform, and by Month 2 you can start adding advanced features like captaincy recommendations, transfer suggestions, and squad optimization.